

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been copied and communicated to you by or on behalf of The University of Newcastle pursuant to Part VB of the Copyright Act 1968 (the Act).

The material in this communication may be subject to copyright under the Act. Any further copying or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.

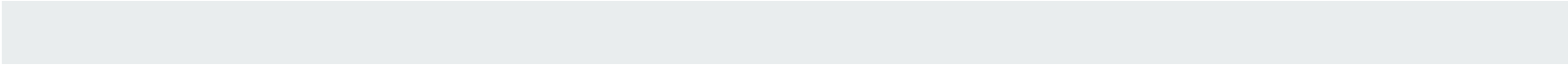
SENG1050

Web Technologies

Week 3 - XML and DTD

Lecture Outline

- ❑ Extensible Markup Language (XML)
- ❑ Structure of an XML Document
- ❑ XML Parsing
- ❑ XML Tree
- ❑ Document Type Definition (DTD)

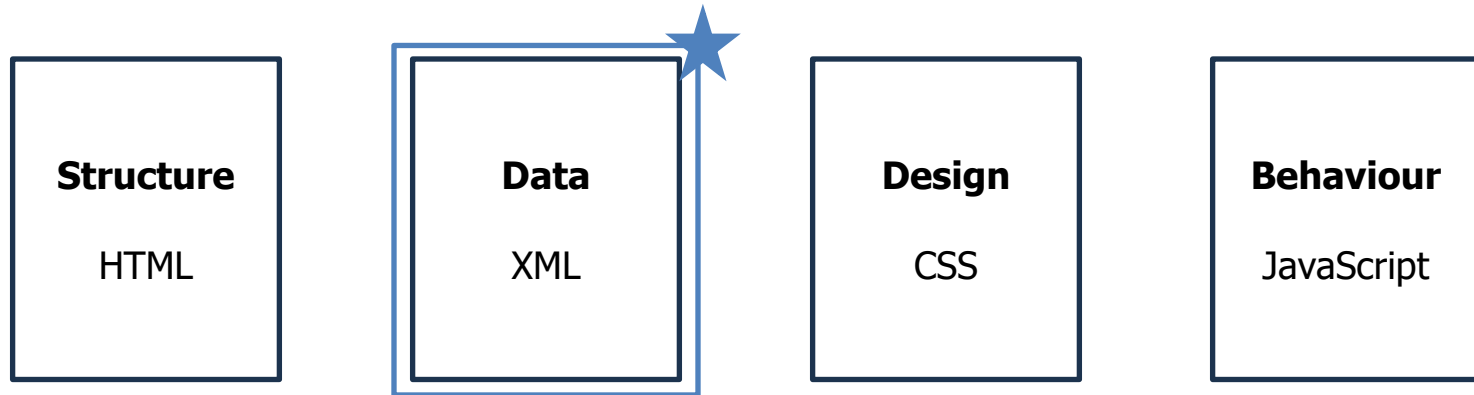


Recall from last week: HTML

- ❑ We have now learned all three elements of the world wide web: the scheme for locating documents (URI), the protocol for accessing them (HTTP), and the language to write them in (HTML).
- ❑ We can write HTML documents to define the structure of our webpages and display data on them.
- ❑ We have two new problems – first, if data ever changes, we would have to update every single webpage using that data (such as product categories on a shopping site). This is inefficient and wastes time.
- ❑ The second problem is that HTML is only intended to specify the structure of a webpage and the semantics of each element. Browsers may visualise these elements in different ways, and so we need a way to explicitly specify visual characteristics of each element to ensure that our website looks the same across all browsers.
- ❑ The solution to the second problem is CSS – next week. We will look at the first problem today.

Extensible Markup Language (XML)

Overview of Client-based Web Development



Dealing with Data

An example of computer vs human interpretation

- ❑ Computer
 - Time 10:30pm
 - Place Sydney
- ❑ Human
 - I need to be on a plane that leaves Sydney at 10:30pm, otherwise I won't get to Los Angeles.

Humans need to know the extra *context* as well as the data.

Can't We Just Use a Database?

While it is true that many modern websites store the bulk of their data in databases, particularly relational databases such as MariaDB and MySQL, XML still has many use cases in its own right:

- ❑ XML is the better choice for transmitting data, database files are many times larger and would therefore take much longer to be sent over a network.
- ❑ For small amounts of data in particular, XML is generally faster to access (no need for database overhead such as when running SQL queries).
- ❑ XML uses considerably less space on disk than an equivalent database.
- ❑ Databases involve more complex management than XML (resource overhead on the server running the database, multi-user access control, regular database maintenance, etc).
- ❑ However, if there is a large amount of data, the data is changing frequently, the data needs to be accessed by multiple writers simultaneously, or access control needs to be implemented for different types of users – in such cases, a database would be the better option.

What is XML?

XML

- ❑ Stands for eXtensible Markup Language
- ❑ A standard set of design rules to make data readable by both humans and computers.
- ❑ A document that follows these rules is an XML Document or an XML Application.

What is eXtensible?

- ❑ It means that the user has full control over the data, context, and patterns used in an XML document.
- ❑ An XML document can contain any user-defined data and structure.
- ❑ XML can be used to exchange data with other users and programs in a platform independent way.
- ❑ XML is self-describing and can be used to express meaningful semantics of documents (it provides context for data).

Think of XML like a **template** or **framework**.

The XML Family

- ❑ XML: eXtensible Markup Language
 - A syntax for arbitrary semantic tags
- ❑ DTD: Document Type Definition
 - Defines the structure (pattern) of a valid XML document
- ❑ XSLT: eXtensible Stylesheet Language Transformations (not covered in course)
 - An xml document that describes how to transform another xml document into a different format.

There are some other members such as XML Schema and XSL-FO that perform similar tasks to DTD and XSLT, but these will not be covered in this course.

Structure of an XML Document



XML Declaration

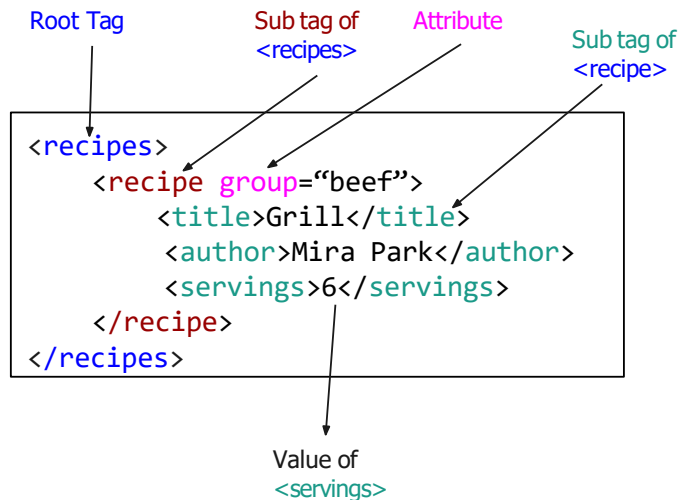
- ❑ Documents begin with XML declaration
`<?xml version="1.0" encoding="utf-8"?>`
- ❑ The declaration must be situated at the **first** position of the **first line** in the **XML** document. This means **before any comments** or even newline characters – the XML declaration must be the literal first line of the document or it will not work correctly.
- ❑ Tells the computer that the file should be read as XML
- ❑ Also defines the encoding that the document is written in.

```
<?xml version="1.0" encoding="utf-8"?>
<departure airport="Newcastle">
  <flight id="001">
    <destination code="LAX">Los Angeles</destination>
    <departureTime>22:30</departureTime>
    <aircraft seats="450">Boeing 757</aircraft>
  </flight>
  <flight id="002">
    <destination code="KTM">Kathmandu</destination>
    <departureTime>23:05</departureTime>
    <aircraft seats="800">Airbus A380</aircraft>
  </flight>
</departure>
```



XML Elements

- ❏ An XML element is everything from (including) the element's opening tag to (including) the closing tag.
- ❏ An element can contain:
 - Root Tag
 - Sub Tag
 - Attributes
 - Element contents (=Data)





How is XML Different from HTML?

XML and HTML were designed with different goals:

- ❑ XML was designed to transport and store data with focus on what data is.
- ❑ HTML was designed to display data, with a focus on how data is structured.
- ❑ Put differently, XML is about the content, whereas HTML is about the format.

XML is not a replacement for HTML

- ❑ It complements HTML

```
<body>
  <h1>About WWF</h1>
  <p>Here is a quote from WWF's website</p>
  <blockquote cite="http://www.worldwildlife.org/who/index.html">
    For 50 years, WWF has been protecting the future of nature.
  </blockquote>
</body>
```

As a sidenote – HTML is case insensitive, whereas XML is case-sensitive. If you've been following best practice with HTML (that is, not mixing cases) this should not be an issue, but it does occasionally cause problems.



XML Element Tags

- ❑ An opening and closing tag (or a content tag)
`<tag> contents </tag>`
- ❑ A single tag
`<tag />` (equivalent to `<tag> ... </tag>` : with no content)
- ❑ Nesting tags (Sub tag inclusion)
`<outerTag>
 <innerTag>Now for some data</innerTag>
</outerTag>`

XML Element Tags Example

- ❑ Tags let us assign a context to each piece of data
 - Example: Write the time 10:30pm in XML

```
<time>10:30pm</time>
<pm>10:30</pm>
<clock>
  <time>10:30</time>
  <amPm>pm</amPm>
</clock>
```
- ❑ Remember, XML is eXtensible - all these answers are correct. The answer you choose is up to you.
- ❑ Can you think of some other answers?

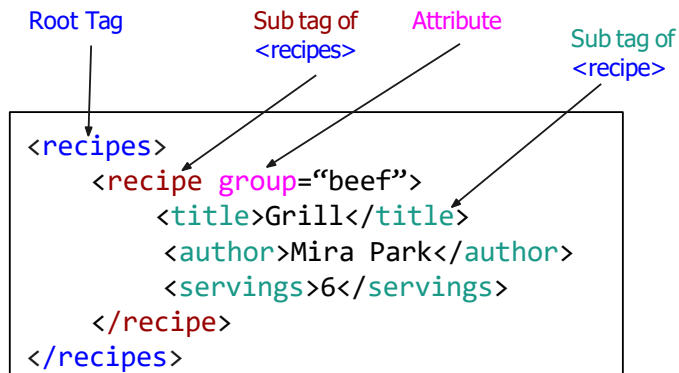


XML Element Attributes

- ❑ All tags optionally contain attributes - small single-word pieces of information that describe a property of the tag
- ❑ Tags can have as many attributes as you like. All attributes must have an associated value, which is given inside quotes.

```
<tag attribute="value"> ... </tag>
```

```
<tag attribute="value" />
```



Value of
<servings>

XML Element Attributes Example

- ❑ You can have as many attributes as you like.
 - Each attribute must have a value

```
<clock time="10:30pm" />
<clock amPm="pm">10:30</clock>
<clock amPm="pm">
    <time hours="10" minutes="30" />
</clock>
```
- ❑ Once again, the attributes you use are totally up to you.



XML Comments

XML applications can contain comments

`<!-- ... -->`

- ❑ These are ignored by the application software
- ❑ Possibly thrown away by the XML Parser

`<!-- remember to fix this -->`

`<!-- think of a better comment -->`

```
<?xml version="1.0" encoding="utf-8"?>
<flights airport="Newcastle">
  <!-- first flight -->
  <departure id="001" time="22:30">
    <destination code="LAX">
      <name>Los Angeles</name>
    </destination>
    <aircraft seats="450">
      <type>Boeing 747</type>
    </aircraft>
  </departure>

  <!-- second flight -->
  <departure id="002" time="23:05">
    <destination code="KTM">
      <name>Kathmandu</name>
    </destination>
    <aircraft seats="800">
      <type>Airbus A380</type>
    </aircraft>
  </departure>
</flights>
```



Rules and Principles

Tag names can be anything you like, provided they follow the following rules:

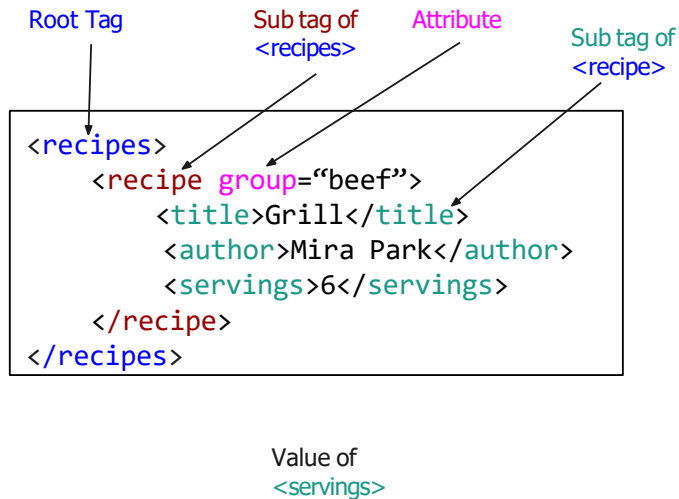
- ❑ Start with a letter or the '_' character
- ❑ Contain only letters, digits, "_", "-", or "."

Tag names are case sensitive!!

- ❑ `<name>` and `<Name>` are NOT the same

Root Tag:

An XML document must have a single container tag that has all other tags and data inside it (except for the XML declaration). This is the root tag.





Rules and Principles

Tags must be correctly nested

- ❑ `<tag>` must have a matching `</tag>`
- ❑ Tags must be opened and closed in the correct order.

❑ **Correct**

```
<foo>
  <bar>
  ...
</bar>
</foo>
```

✗ **Incorrect**

```
<foo>
  <bar>
  ...
</foo>
</bar>
```

- ❑ All XML elements must have a closing tag
`<sample>`
`</sample>`
- ❑ Empty elements must be written as
`<sample></sample>` OR
`<sample />`

Note: XML declaration does not have a closing tag. This is not an error. The declaration is not a part of the XML document itself, and it has no closing tag.

```
<?xml version="1.0" encoding="utf-8"?>
```



A Sample XML document

Consider the following scenario:

- ❑ I want to know
 - All flights leaving Newcastle airport
 - What time they leave
 - Their destination
 - The type of aircraft
 - How many seats are on the aircraft
- ❑ Here is an example XML document for this data

```
<?xml version="1.0" encoding="utf-8"?>
<departure airport="Newcastle">
  <flight id="001">
    <destination code="LAX">Los Angeles</destination>
    <departureTime>22:30</departureTime>
    <aircraft seats="450">Boeing 757</aircraft>
  </flight>
  <flight id="002">
    <destination code="KTM">Kathmandu</destination>
    <departureTime>23:05</departureTime>
    <aircraft seats="800">Airbus A380</aircraft>
  </flight>
</departure>
```

Is there another way you can represent the same data? YES

A Sample XML document

```
<?xml version="1.0" encoding="utf-8"?>
<departure airport="Newcastle">
  <flight id="001">
    <destination code="LAX">Los Angeles</destination>
    <departureTime>22:30</departureTime>
    <aircraft seats="450">Boeing 757</aircraft>
  </flight>
  <flight id="002">
    <destination code="KTM">Kathmandu</destination>
    <departureTime>23:05</departureTime>
    <aircraft seats="800">Airbus A380</aircraft>
  </flight>
</departure>
```

```
<?xml version="1.0" encoding="utf-8"?>
<flights airport="Newcastle">
  <departure id="001" time="22:30">
    <destination code="LAX">
      <name>Los Angeles</name>
    </destination>
    <aircraft seats="450">
      <type>Boeing 747</type>
    </aircraft>
  </departure>
  <departure id="002" time="23:05">
    <destination code="KTM">
      <name>Kathmandu</name>
    </destination>
    <aircraft seats="800">
      <type>Airbus A380</type>
    </aircraft>
  </departure>
</flights>
```

Which of the following functions does an XML document perform?

An XML document stores raw data for use across multiple webpages or websites.

0%

An XML document defines the structure of a webpage and its data.

0%

An XML document stores data with semantic markup in a format that is readable by humans and comp...

0%

An XML document specifies how data should be displayed on a given webpage.

0%

None of the above

0%



Which of the following tag names are invalid in XML?

__name

0%

telephone-num

0%

1stplacewinner

0%

_.--N-A.M-E__

0%

None of the above

0%



XML Parsing

XML Parser

In computing, a parser is a program (or a piece of code) which analyses files to identify the component parts.

- ❑ Parsed data - becomes part of the XML file.
- ❑ Unparsed data - does not become a part of the XML file (e.g., images).

Parsing an XML Document

Most browsers include a built-in XML parser.

- ❑ But don't expect XML files to be displayed as HTML pages.

If there are no syntax errors, the XML document will be displayed.

- ❑ Most browsers will also allow you to collapse or expand the element structure

```
▼ <note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend</body>
</note>
```

```
► <note>
  ...
</note>
```

Parsed Character Data (PCDATA)

Parsed Character Data or PCDATA consists of all those characters that XML treats as parts of the code of an XML document.

- ❑ The XML declaration
- ❑ The opening and closing tags of an element
- ❑ Empty element tags
- ❑ Character or entity references
- ❑ Comments
- ❑ Processing instructions

Unparsed Data (CDATA)

- ❑ Imagine an XML page with large chunk of C code.
- ❑ XML parser will try to parse everything including characters like `&` and `<`, which will result in syntax errors.
- ❑ **CDATA - (Unparsed) Character Data**
- ❑ The term CDATA is used about text data that should not be parsed by the XML parser.
- ❑ Everything inside a CDATA section is ignored by the parser.

- ❑ You can enter raw character data using `<![CDATA[ raw text ]]>`
- ❑ The raw text is parsed through XML parser without processing
- ❑ Example:
`<![CDATA[ foo<bar & bar>foobar ]]>`

In parsed data, equivalent to (foo<bar & bar>foobar) as Unicode entities.

PCDATA vs. CDATA Example

```
<?xml version="1.0" encoding="utf-8"?>

<calculator>
  <boolean>5 < 7 & 4 > 2</boolean>
</calculator>
```



This page contains the following errors:

error on line 4 at column 14: StartTag: invalid element name

Below is a rendering of the page up to the first error.

```
<?xml version="1.0" encoding="utf-8"?>

<calculator>
  <boolean>5 &lt; 7 &amp; 4 &gt; 2</boolean>
  <boolean><![CDATA[5 < 7 & 4 > 2]]></boolean>
  <boolean>&#189;</boolean>
</calculator>
```



```
▼<calculator>
  <boolean>5 < 7 & 4 > 2</boolean>
  ▼<boolean>
    <![CDATA[ 5 < 7 & 4 > 2 ]]>
  </boolean>
  <boolean>½</boolean>
</calculator>
```

Which type of data should be used for displaying long blocks of computer code?

element data

0%

Parsed character data

0%

Unparsed character data

0%

Preformatted text data

0%

None of the above

0%



XML Tree



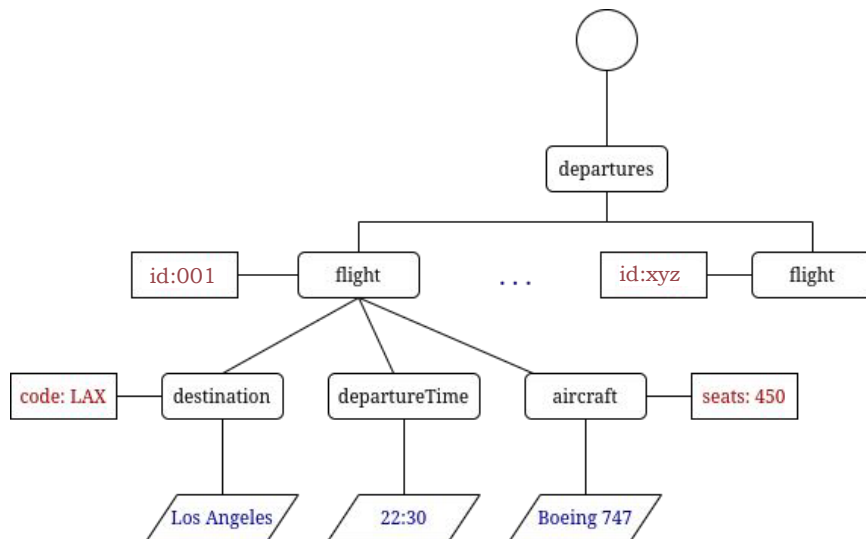
XML Trees

- ❑ Conceptually, an XML document is a hierarchical structure called an XML tree, consisting of nodes of various kinds arranged as a tree
- ❑ XML Trees: Another way to represent XML that may make it easier to understand
- ❑ Let us visualise the structure.



XML Tree

```
<?xml version="1.0" encoding="utf-8"?>
<departures>
  <flight id="001">
    <destination code="LAX">Los Angeles</destination>
    <departureTime>22:30</departureTime>
    <aircraft seats="450">Boeing 747</aircraft>
  </flight>
  <flight id="002">
    <destination code="KTM">Kathmandu</destination>
    <departureTime>23:05</departureTime>
    <aircraft seats="800">Airbus A380</aircraft>
  </flight>
</departures>
```





XML Trees Rules and Principles

Recall

- ❑ All elements must be nested within a single document or root tag.
- ❑ There can only be one root tag.

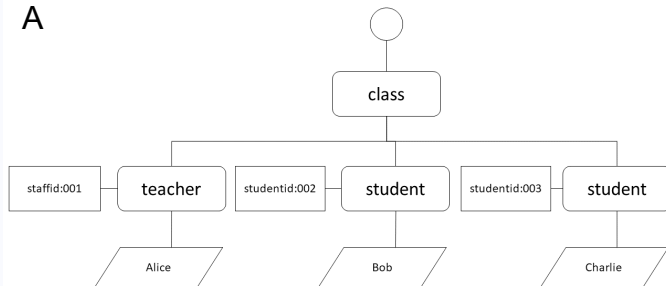
Which of the following XML trees matches the given XML?

```
<class>  
  <teacher staffid="001">Alice</teacher>  
  <student studentid="002">Bob</student>  
  <student studentid="003">Charlie</student>  
</class>
```

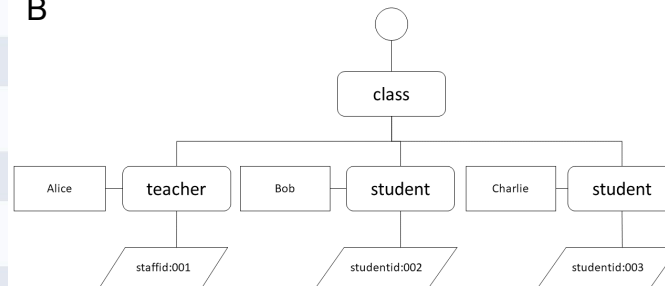
A

0%

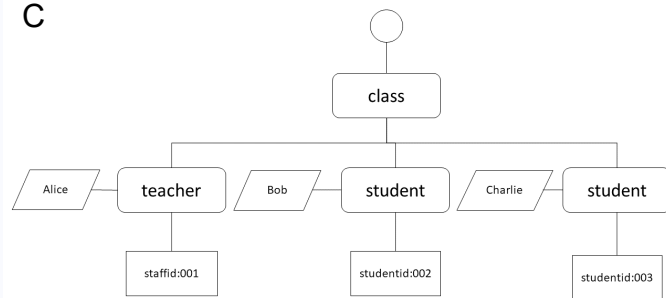
A



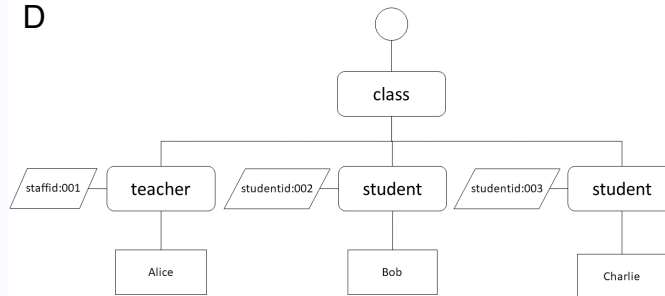
B



C



D



Document Type Definition (DTD)

How to Define our Own Tags

DTD (Document Type Definition)

- ❑ Defines the structure (pattern) of a valid XML document and with a list of legal elements and attributes.
- ❑ The purpose of a DTD is to define the legal building blocks of an XML document.

XML Schema

- ❑ XML Schema is an XML-based alternative to DTDs.
- ❑ **Not covered in this course.**



Document Type Definition (DTD)

A DTD describes:

- ❑ Which tags are valid in an XML document.
- ❑ Which tags can be nested inside which tags.
- ❑ Which attributes are valid for which tags.

DTD Types

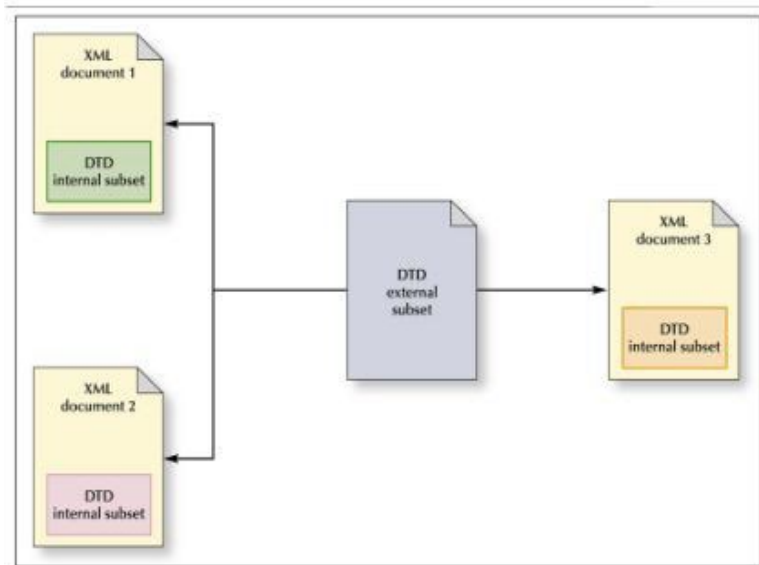
It can be divided into two types:

Internal DTD:

The declarations placed in the same file as the document content.

External DTD:

Located in a separate file





Linking XML and DTD

Internal DTD

```
<!DOCTYPE root_tag [  
    DTD_rules  

```

External DTD

```
<!DOCTYPE root_tag SYSTEM "file">
```



Specifying XML Element Tags in DTD

`<!ELEMENT element-name (children)>`

- ❑ **children** is a list of tags; commas (,) separate items in the list.
`<!ELEMENT family (mother,father,child)>`
`<!ELEMENT body (shape)>`
- ❑ Vertical bars (|) separate "choices"
`<!ELEMENT foo (bar|foobar|baz)>`
- ❑ Children should appear in the same order in the document, as they were declared in the sequence

`<!ELEMENT element-name category>`

- ❑ Empty elements are declared with the category keyword **EMPTY**:
`<!ELEMENT img EMPTY>`
-> allows `<img ... />` to be valid
- ❑ Elements with only parsed character data are declared with **#PCDATA** inside parentheses
`<!ELEMENT departureTime (#PCDATA)>`
- ❑ Elements with the **ANY** keyword can contain any combination of parsable data.
`<!ELEMENT departureTime ANY>`



Specifying XML Elements Tags in DTD With Modifiers

- ❑ Each child has an optional modifier
 - **child** one and only one such child allowed
 - **child+** one or more such children allowed
 - **child*** zero or more such children allowed
 - **child?** Zero or one such children allowed
- ❑ Mixed content
((#PCDATA| *child2* | ... | *childn*)*)

Examples

```
<!ELEMENT body (shape)>
<!ELEMENT departureTime (#PCDATA)>
<!ELEMENT family (parent?,parent?,child*)>
<!ELEMENT parent (firstName,surname,DOB)>
...
<!ELEMENT foo (bar+|foobar*|baz?)>
<!ELEMENT anything (#PCDATA|name|note)*>
```



Specifying XML Element Attributes in DTD

```
<!ATTLIST element-name attribute-name attribute-type default-value>
```

Example

```
<!ELEMENT circle EMPTY>  
<!ATTLIST circle radius CDATA "0">
```

Valid XML

```
<circle radius="10">
```



Specifying XML Element Attributes in DTD

`<!ATTLIST element-name attribute-name attribute-type default-value>`

Some commonly used attribute-types are:

Type	Description
<code>CDATA</code>	The value is character data
<code>(en1 en2 ..)</code>	The value must be one of the listed values
<code>ID</code>	The value is an unique id
etc ...	

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE image [
  <!ELEMENT image EMPTY>
  <!ATTLIST image height CDATA #REQUIRED>
  <!ATTLIST image width CDATA #REQUIRED>
]>
<image height="32" width="32" />
```

Specifying XML Element Attributes in DTD Example

```
<!DOCTYPE payment [  
  <!ELEMENT payment (#PCDATA)>  
  <!ATTLIST payment method (cheque|cash) "cash">  
  
<payments>  
  <!-- Valid XML -->  
  <payment method="cheque" />  
  <payment method="cash" />  
  
  <!-- Invalid XML -->  
  <payment method="credit" />  
</payments>
```



Specifying XML Element Attributes in DTD

```
<!ATTLIST element-name attribute-name attribute-type default-value>
```

Possible options for default-value

value	The default value
#FIXED value	Value cannot be changed by the tag
#REQUIRED	Must be supplied by the tag
#IMPLIED	No default, but doesn't have to be supplied by the tag



Specifying XML Element Attributes in DTD

`<!ATTLIST element-name attribute-name attribute-type default-value>`

default = value

DTD

```
<!ELEMENT circle EMPTY>
<!ATTLIST circle radius CDATA "0">
```

Valid XML

```
<circle radius="100" />
```

Valid XML

```
<circle />
```

If no radius is specified, it has a default radius of 0

default = #FIXED value

DTD

```
<!ATTLIST sender company CDATA #FIXED "Microsoft">
```

Valid XML

```
<sender company="Microsoft" />
```

Invalid XML

```
<sender company="W3Schools" />
```

Use the #FIXED keyword when you want an attribute to have a fixed value. If another value is given, the parser will return an error.



Specifying XML Element Attributes in DTD

`<!ATTLIST element-name attribute-name attribute-type default-value>`

default = #REQUIRED

DTD

`<!ATTLIST person number CDATA #REQUIRED>`

Valid XML

`<person number="5677" />`

Invalid XML

`<person />`

It forces that the attribute must always be included

default = #IMPLIED

DTD

`<!ATTLIST contact fax CDATA #IMPLIED>`

Valid XML

`<contact fax="555-667788" />`

Valid XML

`<contact />`

Use the #IMPLIED keyword if you don't want to force the inclusion of an attribute.



Specifying XML Element Entities in DTD

Entities

- Entities are variables used to define shortcuts to standard text or special characters.
- Entity references are references to entities.

As well as XML predefined entities such as `>`, `<`, `&`, `"`, `'`, other entities can be defined in DTD.



Specifying XML Element Entities in DTD

- ❑ `<!ENTITY name "value">`
 - defines a new character entity `&name;`
- ❑ For example, an entity named "MBL25" can be created to store a product description:
`<!ENTITY MBL25 "Monarch Butterfly, 6-12 larvae">`
- ❑ After an entity is declared, it can be referenced anywhere within the document.
`<item>&MBL25;</item>`
- ❑ This is interpreted as:
`<item>Monarch Butterfly, 6-12 larvae </item>`



XML + DTD Example

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE greeting [
  <!ELEMENT greeting (#PCDATA)>
  <!ATTLIST greeting style (big|small) "small">
  <!ENTITY hi "Hello">
]>
<greeting style="big"> &hi; world! </greeting>
```

```
<greeting style="big"> Hello world! </greeting>
```



XML + DTD Example

```
<?xml version="1.0" encoding="utf-8"?>

<!DOCTYPE note [
  <!ELEMENT note (to,from,heading,body)>
  <!ELEMENT to (#PCDATA)>
  <!ELEMENT from (#PCDATA)>
  <!ELEMENT heading (#PCDATA)>
  <!ELEMENT body (#PCDATA)>
]>

<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend</body>
</note>
```

```
▼ <note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend</body>
</note>
```



XML Validation Against DTD

- ❏ A document is **well-formed** XML if-and-only-if:
 - Its syntax conforms to the XML specification
 - Its tags form a hierarchical tree, with a single root node.

- ❏ A document is **valid** XML if-and-only-if:
 - It is **well-formed**, and
 - It **conforms to an associated DTD**
 - <http://www.xmlvalidation.com>

Which of the following is the correct way to define an attribute named 'colour' for a tag named 'hair' with a default value of 'cyan' that may be optionally overridden in XML?

< !ATTLIST colour hair "cyan" #IMPLIED>

0%

< !ATTLIST colour hair "cyan" #IMPLIED>

0%

< !ATTLIST hair colour CDATA "cyan">

0%

< !ATTLIST colour hair CDATA #FIXED "cyan">

0%

None of the above

0%



Given the DTD declaration `<!ENTITY mycopy "Copyright 2022">`, which of the following XML elements correctly reference the entity?

`< author>&mycopy< /author>`

0%

`< author>Copyright 2022< /author>`

0%

`< author>mycopy< /author>`

0%

`< author>&mycopy;< /author>`

0%

None of the above

0%



Which of the following XML block would be valid for the given DTD?

```
<!DOCTYPE sets
[
  <!ELEMENT sets(set+)>
  <!ELEMENT set(setA | setB)>
  <!ELEMENT setA((setB*, value+|setA)+)>
  <!ELEMENT setB(setA, setB*)>
  <!ELEMENT value(#PCDATA)>
]>
```

```
<sets>
</sets>
```

A

```
<sets>
  <set></set>
  <set></set>
</sets>
```

B

```
<sets>
  <set>
    <setA>
      <setB>
        <setA>
          <value>404</value>
        </setA>
      </setB>
      <value>WHAT</value>
    </setA>
  </set>
</sets>
```

C

```
<sets>
  <set>
    <setB>
      <setA>
        <value>404</value>
      <setA>
        <setB>
          <setA>
            <value>HELP</value>
          </setA>
        </setB>
      </setB>
    <setA>
      </setA>
    </setB>
  </set>
</sets>
```

0%

0%

D



Which of the following conditions must be satisfied for an XML document to be considered well-formed? Select the TWO correct answers.

The document must have a single root element.

0%

The document must conform to the associated DTD.

0%

The document has passed an online validator with no errors detected.

0%

The document must begin with an XML declaration.

0%

The document must specific either an internal or external (or both) DTD.

0%





Summary

- ❑ We have now addressed the issue of separating the structure of our website from the data. This means we now have only one file to update when data changes instead of multiple files.
- ❑ We have learned to read and write our own XML documents, using a DTD to define our own custom tags. This will allow us to store any arbitrary data using whatever structure we need, in a manner that is simple for both humans and computers to read.
- ❑ We now turn our attention to the second problem we identified with HTML – specifying visual styling for elements to ensure consistency across different browsers. To achieve this we will use CSS – next week.



Summary

- ☐ The rules and principles of XML
- ☐ XML Parsing
 - CDATA and PCDATA
- ☐ XML Trees - the hierarchical structure of the XML document
- ☐ How to define your own XML tags
- ☐ Internal and External DTD
- ☐ How to declare DTD elements
- ☐ How to set the attributes for DTD elements
- ☐ What is a DTD entity